

CR – TP6 – RX

1. Cahier des charges de l'application

Objectif général :

Développer une application en C qui implémente le jeu du Mastermind en mode client/serveur, en utilisant la famille de protocoles TCP-IP via les sockets.

Fonctionnalités principales attendues :

- **Le client :**
 - Se connecte au serveur en utilisant une adresse IP et un port spécifié.
 - Saisit un niveau de jeu qui détermine le nombre de positions (ou emplacements) à deviner.
 - Saisit des propositions, sous forme d'une chaîne de chiffres séparés par des espaces (chaque chiffre représentant une couleur codée de 1 à 8).
 - Affiche la réponse du serveur qui indique :
 - Le nombre de couleurs correctement placées (*Red*).
 - Le nombre de couleurs présentes dans la solution mais mal positionnées (*White*).
 - Propose à la fin de la partie, qu'elle soit gagnée ou interrompue, de rejouer.
- **Le serveur :**
 - Se met en écoute sur un port défini (par défaut 1111) et accepte les connexions entrantes.
 - Gère chaque connexion dans un mode parallèle si on opère en mode parallèle sinon en mode itératif (en créant un processus enfant par connexion grâce à `fork()`).
 - Pour chaque client, lit la commande « LEVEL n » envoyée par le client afin d'en déduire la longueur de la combinaison secrète.
 - Génère aléatoirement un code secret (tableau d'entiers) où chaque chiffre est compris entre 1 et 8.
 - Pour chaque proposition envoyée par le client, compare la proposition avec le code secret et renvoie une réponse sous la forme « Red: x, White: y » (x étant le nombre d'éléments parfaitement placés et y le nombre d'éléments présents dans la solution mais à une position différente).
 - À la fin d'une partie, propose de rejouer et, en cas de refus du client, ferme la connexion pour ce client (sans affecter les autres sessions).

Options supplémentaires :

- Gérer plusieurs clients simultanément (serveur parallèle).

2. Solutions choisies et argumentations

Dans le cadre de ce TP, nous avons apporté plusieurs solutions techniques pour répondre aux exigences fonctionnelles et garantir une expérience utilisateur cohérente. Voici les principales décisions que nous avons prises durant le développement, ainsi que leurs justifications :

1. Gestion des entrées et validation du format

Afin d'éviter des incohérences liées aux saisies erronées par l'utilisateur, nous avons implémenté une fonction de validation du format des propositions.

- Solution apportée* : La fonction `validate_proposal()` vérifie que, pour un niveau supérieur à 1, les chiffres sont bien séparés par des espaces et que le nombre de tokens correspond exactement au nombre de positions attendu.
- Argumentation* : Cette validation empêche le traitement de propositions mal formatées qui pourraient fausser le résultat du jeu (par exemple, des chiffres collés ou un nombre incorrect de chiffres).

2. Affichage amélioré grâce aux codes ANSI

Pour rendre l'interface utilisateur plus lisible, nous avons intégré des codes de couleurs ANSI afin d'afficher par exemple la partie "Red:" en rouge.

- Solution apportée* : La fonction `print_response_with_color()` découpe la chaîne de réponse du serveur pour mettre en évidence le segment "Red:" en rouge.
- Argumentation* : Cette solution améliore la lisibilité des retours du serveur et permet à l'utilisateur de distinguer rapidement les informations importantes (positions correctes) dans la réponse.

3. Gestion de la session de jeu en mode client

Le client doit pouvoir jouer plusieurs parties sur une même connexion.

- Solution apportée* : Le client dispose d'une boucle de session dans laquelle il saisit le niveau du jeu, envoie la commande correspondante au

serveur, puis envoie ses propositions. Une fois la partie terminée (par victoire ou par décision de quitter), il peut choisir de rejouer ou non.

- b. *Argumentation* : Cela permet d'éviter de devoir se reconnecter à chaque partie et offre une meilleure continuité dans l'expérience de jeu.

4. Implémentation d'un serveur itératif et parallèle

Pour le serveur, nous avons développé deux modes d'architecture afin de satisfaire différents scénarios :

- a. *Mode itératif* : Le serveur traite une connexion à la fois. Ce mode est simple et convient aux tests en environnement contrôlé.
- b. *Mode parallèle* : Le serveur crée un processus enfant via `fork()` pour chaque client.

Solution apportée : Dans la version parallèle, nous utilisons `fork()` immédiatement après l'acceptation d'une connexion pour que chaque client soit géré indépendamment dans son propre processus. Pour éviter les processus zombies, nous avons configuré le signal `SIGCHLD` avec `signal(SIGCHLD, SIG_IGN)`.

Argumentation : Le mode parallèle permet à plusieurs clients de jouer simultanément sans qu'une session n'interfère avec l'autre. Il s'agit d'une solution robuste pour un environnement multi-utilisateurs,

5. Utilisation d'une boîte à outils sur les sockets (`fon.c/fon.h`)

L'utilisation de fonctions d'encapsulation (comme `h_socket()`, `h_bind()`, etc.) simplifie le développement et la gestion des erreurs, tout en assurant une portabilité entre plusieurs systèmes. Cependant, dans notre implémentation finale, nous avons choisi d'utiliser la fonction "`read_line`" qu'on a implémenté nous-même à la place de "`h_read`". Comme cela, les joueurs peuvent saisir leurs tentatives sur une seule ligne qui rend l'interface plus lisible et confortable.

3. Description des protocoles utilisés dans votre application

Protocole de communication (dialogue client/serveur) :

1. Établissement de la connexion :

- a. **Client → Serveur** : Le client se connecte à l'aide d'une socket TCP sur le port défini (par défaut, 1111) en utilisant l'adresse IP du serveur.
- b. **Serveur** : Le serveur accepte la connexion et envoie immédiatement un message de bienvenue sous la forme :

Bienvenue dans Mastermind !

2. Début de la session de jeu :

- a. **Client → Serveur** : Le client envoie une commande « LEVEL n » pour indiquer le nombre de positions (n) du jeu. Exemple :

LEVEL 4

- b. **Serveur** : Le serveur lit cette commande et, à partir du paramètre, génère un code secret aléatoire composé de n chiffres (chacun compris entre 1 et 8).

3. Échange lors de la partie :

- a. **Client → Serveur** : Le client envoie une proposition, attendue sous forme d'une séquence de chiffres séparés par des espaces. Exemple pour un niveau 4 :

1 4 7 3

- b. **Serveur** :
Pour chaque proposition, le serveur compare la proposition au code secret et calcule deux valeurs :
 - i. **Red**: Le nombre de chiffres présents à la bonne position.
 - ii. **White**: Le nombre de chiffres présents dans la solution mais à une position différente.Le serveur répond alors par une ligne de texte, par exemple :

Red: 2, White: 1

ou, en cas de victoire (proposition correcte) :

GAGNE! Red: 4, White: 0

4. Fin de partie et proposition de rejouer :

- a. **Serveur → Client** : Une fois la partie terminée (que ce soit par victoire ou par arrêt du client), le serveur envoie une question pour demander si le client souhaite rejouer :

Voulez-vous rejouer ? (o/n) :

- b. **Client → Serveur** : Le client répond par "o" (ou "O") pour continuer ou par une autre lettre pour terminer la session.
Le serveur traitera la réponse en démarrant une nouvelle partie pour ce client (dans le processus enfant) ou en fermant la connexion si la réponse est négative.

5. Gestion de la session parallèle :

- a. Chaque client se voit attribuer un processus enfant sur le serveur. Les échanges se font indépendamment pour chaque connexion, ce qui permet à plusieurs clients de jouer simultanément sans interférer les uns avec les autres.

Aspects complémentaires

- **Fiabilité et Contrôle d'erreur** : L'utilisation de TCP permet d'assurer que chaque message envoyé sera effectivement reçu, ce qui est primordial pour le bon déroulement du jeu. En cas de déconnexion ou d'erreur, le serveur peut interrompre la session de jeu et informer le client.
- **Simplicité du protocole textuel** : Le choix de communiquer en texte (au lieu d'un format binaire ou d'utiliser UDP) simplifie le développement, le débogage et la maintenance de l'application.
- **Indépendance des sessions en mode parallèle** : Avec l'utilisation de fork() dans la version parallèle, le serveur garantit que chaque connexion est traitée indépendamment, permettant à plusieurs clients de jouer simultanément sans interférence.

Serveur Itérative



